

MANUAL TÉCNICO · DOC. DO SISTEMA



HOT GARAGE SKULL CREW

DOCUMENTAÇÃO DO SISTEMA WEB

Sistema de gestão para operação de oficina mecânica

KEEP IT HOT.

Frontend React · API REST Node.js/Express · MySQL · JWT

01 SEÇÃO 01 VISÃO GERAL

O **Oficina Mecânica · Hot Garage Skull Crew** é um sistema web desenvolvido para apoiar a operação de uma oficina mecânica. O projeto reúne uma aplicação frontend em React para gerenciamento da oficina e uma API REST desenvolvida em Node.js e Express, com persistência de dados em MySQL e autenticação baseada em token JWT.

A aplicação permite administrar clientes, veículos e ordens de serviço, além de disponibilizar autenticação, dashboard e perfil da conta.

02 SEÇÃO 02 FUNCIONALIDADES

// 01

LOGIN E REGISTRO

Permite autenticar usuários e realizar o registro de novas contas.

// 02

DASHBOARD

Apresenta uma visão geral de clientes, veículos e ordens de serviço.

// 03

CLIENTES

Cadastro, consulta, edição e remoção de clientes.

// 04

VEÍCULOS

Cadastro, consulta, edição e remoção de veículos vinculados a clientes.

// 05

ORDENS DE SERVIÇO

Gestão de cliente, veículo, descrição, valor, data e status.

// 06

PERFIL

Exibe dados da conta e permite encerrar a sessão.

// 07

ROTAS PRIVADAS

Restringe o acesso às áreas autenticadas do sistema.

// 08

TOKEN BEARER

Envia automaticamente o token de autenticação nas chamadas à API.

03 TECNOLOGIAS UTILIZADAS

CAMADA	TECNOLOGIAS
FRONTEND UI / CLIENT	React, Vite, React Router, Tailwind CSS, Axios
BACKEND API / SERVER	Node.js, Express, MySQL2
AUTENTICAÇÃO AUTH / TOKEN	JWT e bcrypt
BANCO DE DADOS PERSISTÊNCIA	MySQL

04 SEÇÃO 04 ESTRUTURA DO PROJETO

■ PROJECT TREE - OFICINA_MECANICA

```
Oficina_mecanica_simulado/  
├── backend/  
│   ├── config/           # Conexão, schema e seed do banco  
│   ├── controller/       # Regras dos endpoints  
│   ├── middlewares/      # Autenticação  
│   ├── routes/           # Rotas da API  
│   └── src/               # Inicialização do servidor e da aplicação  
└── frontend/  
    ├── oficina_frontend/  
    │   └── src/  
    │       ├── components/ # Componentes visuais  
    │       ├── lib/        # Estado da oficina e utilitários  
    │       ├── pages/      # Telas do sistema  
    │       ├── routes/     # Rotas do frontend  
    │       └── services/    # Comunicação com a API
```

4.1 BACKEND

config: concentra os arquivos relacionados à configuração do banco, incluindo conexão, schema e seed.

controller: contém as regras associadas aos endpoints. **middlewares:** concentra o mecanismo de autenticação.

routes: organiza as rotas da API. **src:** contém a inicialização da aplicação e do servidor.

4.2 FRONTEND

components: componentes visuais reutilizáveis. **lib:** estado da oficina e utilitários. **pages:** telas do sistema.

routes: rotas da aplicação frontend. **services:** comunicação com a API.

NODE.JS

20.19+ ou 22.12+

Node.js 20.19+ ou 22.12+ e npm.

MYSQL

8 ou compatível

MySQL 8 ou compatível.

GIT

opcional

Caso o projeto seja clonado de um repositório.

06

SEÇÃO 06

CONFIGURAÇÃO DO BANCO DE DADOS

O projeto utiliza **MySQL** como banco de dados. O schema do projeto cria e seleciona o banco **saep_db**. O arquivo de schema pode ser executado pelo terminal ou aberto diretamente no MySQL Workbench.

EXECUÇÃO PELO POWERSHELL

■ POWERSHELL

```
PS> Get-Content -Raw backend/config/schema.sql | mysql -u root -p
```

Também é possível abrir **backend/config/schema.sql** no MySQL Workbench e executar o arquivo.

07

SEÇÃO 07

CONFIGURAÇÃO E EXECUÇÃO DA API

7.1 ARQUIVO DE AMBIENTE

■ POWERSHELL

```
PS> Copy-Item backend/.env.example backend/.env
```

VARIÁVEL	DESCRIÇÃO	PADRÃO DO EXEMPLO
PORT	Porta HTTP da API	3000
DB_HOST	Servidor MySQL	localhost
DB_USER	Usuário MySQL	root
DB_PASSWORD	Senha MySQL	senai
DB_NAME	Banco de dados	saep_db
JWT_SECRET	Chave de assinatura dos tokens	Troque por uma chave privada

■ POWERSHELL

```
PS> Set-Location backend
PS> npm ci
PS> npm run seed
PS> npm run dev
```

A API fica disponível em **http://localhost:3000**. Para iniciar sem o modo de desenvolvimento, utilize **npm start**.

O seed cria o usuário inicial **admin@oficina.com** com a senha **12345678**. O README recomenda alterar essa senha assim que possível e evitar essas credenciais fora de um ambiente local.

08

SEÇÃO 08

INICIALIZAÇÃO DO FRONTEND

■ POWERSHELL

```
PS> Set-Location frontend/oficina_frontend
PS> npm ci
PS> npm run dev
```

O frontend é executado pelo **Vite**, normalmente em **http://localhost:5173**. Em desenvolvimento, o Vite encaminha as chamadas **/api** para **http://localhost:3000**.

Para alterar o destino do proxy de desenvolvimento, pode ser utilizada a variável **VITE_API_PROXY_TARGET**. Em builds servidos em produção, a variável **VITE_API_URL** define a URL base da API, incluindo **/api**.

09

SEÇÃO 09

ROTAS DO FRONTEND

CAMINHO	TELA	ACESSO
/	Login	PÚBLICO
/dashboard	Painel da oficina	AUTENTICADO
/clientes	Clientes	AUTENTICADO
/veiculos	Veículos	AUTENTICADO
/ordens-servico	Ordens de serviço	AUTENTICADO
/profile	Perfil	AUTENTICADO

SEÇÃO 10

As rotas de dados exigem o cabeçalho de autorização no formato **Authorization: Bearer <token>**.

MÉTODO	ENDPOINT	DESCRIÇÃO
POST	/api/auth/login	Autentica e retorna token e usuário
POST	/api/auth/register	Registra usuário
GET	/api/clientes	Lista clientes
POST	/api/clientes	Cria cliente
GET PUT DELETE	/api/clientes/:id	Consulta, atualiza ou remove cliente
GET	/api/veiculos	Lista veículos
POST	/api/veiculos	Cria veículo
GET PUT DELETE	/api/veiculos/:id	Consulta, atualiza ou remove veículo
GET	/api/ordens-servico	Lista ordens de serviço
POST	/api/ordens-servico	Cria ordem de serviço
GET PUT DELETE	/api/ordens-servico/:id	Consulta, atualiza ou remove ordem

10.1 EXEMPLO DE AUTENTICAÇÃO

■ HTTP REQUEST

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "admin@oficina.com",
  "password": "12345678"
}
```

II COMANDOS ÚTEIS

FRONTEND – EXECUTAR EM FRONTEND/OFICINA_FRONTEND

■ NPM SCRIPTS – FRONTEND

```
npm run dev      # Servidor local
npm run build     # Build de produção
npm run preview   # Visualiza o build localmente
npm run lint      # ESLint
```

BACKEND – EXECUTAR EM BACKEND

■ NPM SCRIPTS – BACKEND

```
npm run dev      # API com reinicialização automática via Node
npm start        # API em modo normal
npm run seed     # Insere os dados de demonstração no banco
```

12

SEÇÃO 12

SEGURANÇA E OBSERVAÇÕES

/// ATENÇÃO – BOAS PRÁTICAS DE SEGURANÇA

backend/.env	O arquivo backend/.env contém credenciais e não deve ser enviado ao repositório.
SEED	O seed deve ser executado uma vez em um banco vazio; os registros utilizam identificadores fixos para relacionar clientes, veículos e ordens.
Expiração token	O token JWT expira após uma hora. Quando a sessão expirar, o usuário deverá realizar o login novamente.
JWT_SECRET	Em produção, o README recomenda utilizar segredos e senhas fortes (troque o JWT_SECRET por uma chave privada).
HTTPS / CORS	Em produção, o README recomenda configurar HTTPS e restringir o CORS ao domínio do frontend.
Senha do seed	Altere a senha padrão (12345678) do usuário admin@oficina.com assim que possível e evite essas credenciais fora de um ambiente local.

13 CONSIDERAÇÕES FINAIS

O sistema integra uma interface web em React com uma API REST em Node.js/Express e um banco de dados MySQL. A arquitetura apresentada separa responsabilidades entre frontend, backend, persistência, autenticação e comunicação com a API, permitindo a gestão dos principais recursos de uma oficina: usuários, clientes, veículos e ordens de serviço.

Esta documentação foi elaborada a partir do README fornecido para o projeto Oficina Mecânica · Hot Garage Skull Crew.

KEEP IT HOT.